



คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Chapter 03 : Introduction to Python for Image and Video Understanding

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรณโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง



หัวข้อ

- แนะนำชุดคำสั่ง OpenCV
- แนะนำ อาร์เรย์ (Array) ในชุดคำสั่ง numpy
 - วิธีการสร้าง
 - วิธีการอ้างอิงตำแหน่ง
 - ตัวดำเนินการทางคณิตศาสตร์
 - การเชื่อมต่ออาร์เรย์
 - การปรับขนาดและคัดลอก
- แนะนำชุดคำสั่ง matplotlib



แนะนำชุดคำสั่ง OpenCV

การเรียกใช้ OpenCV ในโปรแกรม Python

รูปแบบที่ 1

```
import cv2
```

รูปแบบที่ 2

```
import cv2 as cv
```




แนะนำชุดคำสั่ง OpenCV

วิธีการนำเข้ารูปภาพ

คำสั่ง

```
ตัวแปร = cv2.imread('ที่อยู่ของรูปภาพ/ชื่อรูปภาพ.นามสกุล')
```

ตัวอย่างเช่น

```
import cv2
img_color = cv2.imread('/content/drive/My Drive/img/lenna_color.png', 1)
img_grayscale = cv2.imread('/content/drive/My Drive/img/lenna_color.png', 0)
```

เพื่อระบุว่าเป็นภาพระดับสีเทา
(grayscale image)

หมายเหตุ ค่าเริ่มต้นของคำสั่ง imread คือ cv2.IMREAD_COLOR หรือใส่เลข 1 ทำให้อ่านภาพมาได้ 3 มิติ หรือเป็นภาพสี แต่ถ้าต้องการอ่านภาพขาวดำหรือภาพระดับสีเทาก็กำหนดเป็น cv2.IMREAD_GRAYSCALE หรือใส่เลข 0



แนะนำชุดคำสั่ง OpenCV

วิธีการแสดงรูปภาพ

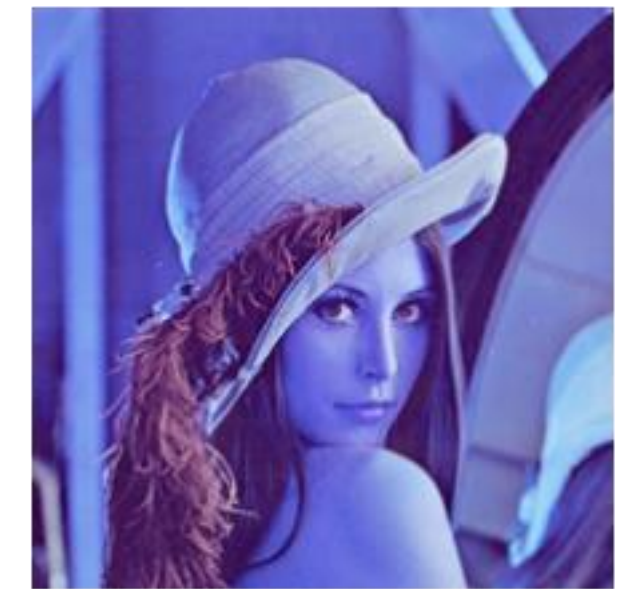
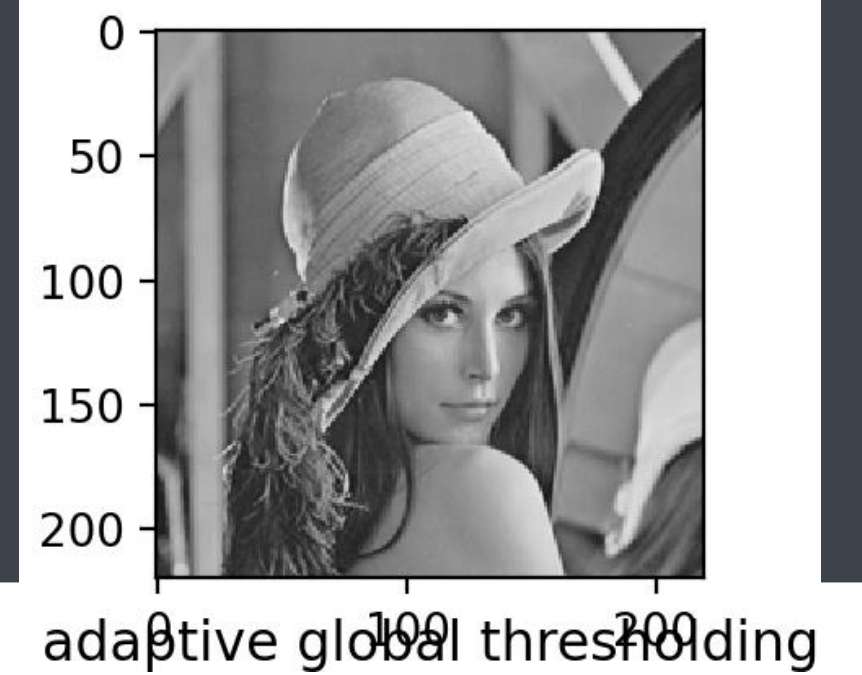
นักศึกษาสามารถแสดงรูปภาพแบบโดยใช้คำสั่ง `imshow()` จากไลบรารี `Matplotlib` ได้

ตัวอย่างเช่น

```
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive') # สร้างต่อเชื่อมต่อระหว่าง Google Drive กับ Colab

img = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)

plt.axis("off") # ไม่อนุญาตให้แสดงเส้นแกน
plt.imshow(img) # คำสั่ง plt.imshow(...) ใช้สำหรับแสดงรูปภาพที่เก็บไว้ในตัวแปร im
```

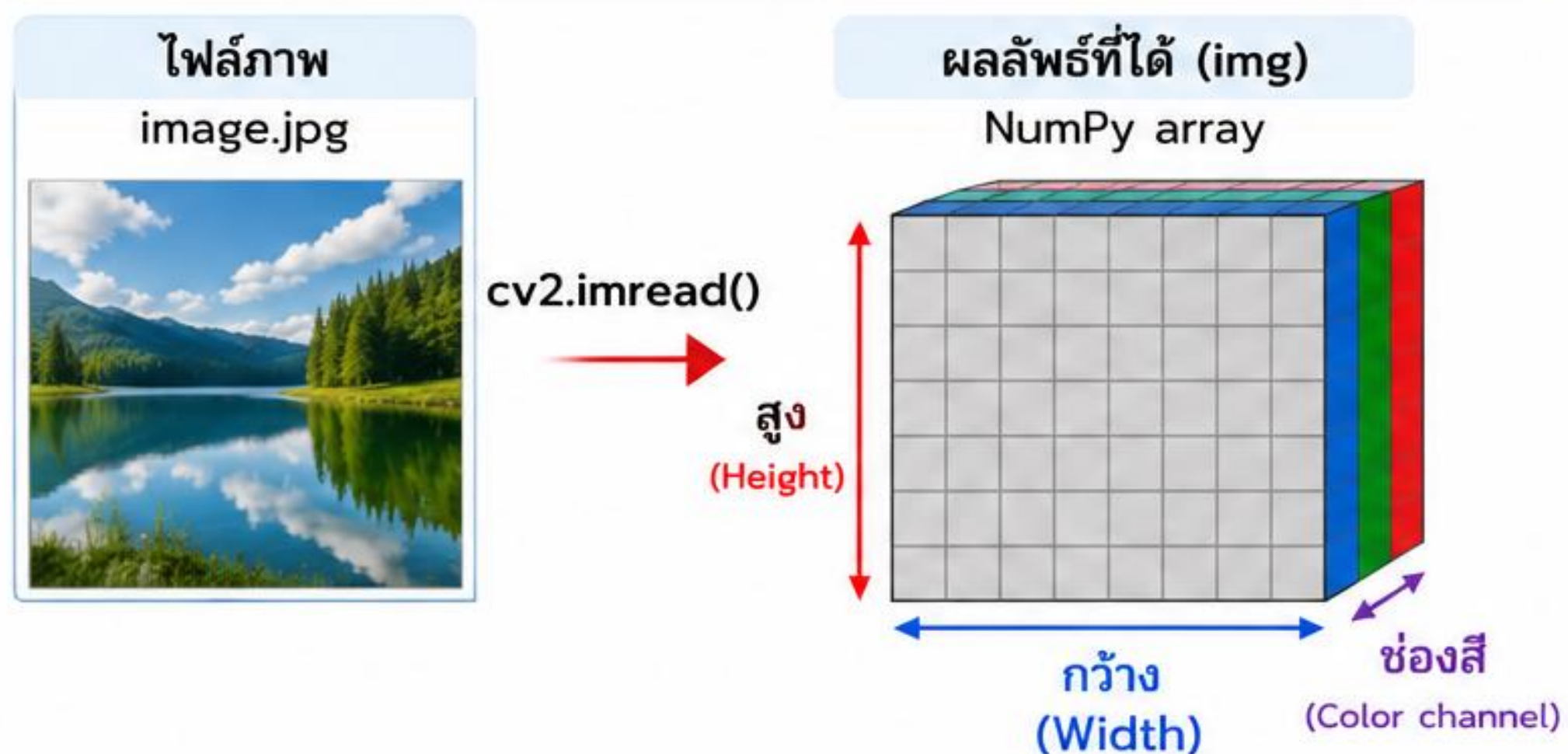




แนะนำชุดคำสั่ง OpenCV

ตัวอย่างการใช้งาน

```
import cv2  
img = cv2.imread('image.jpg')
```



ชนิดข้อมูล (data type)

- โดยทั่วไปจะเป็น uint8
คือ จำนวนเต็มไม่ติดลบ 8 บิต มีค่าตั้งแต่ 0 ถึง 255
- 0 ————— 255
- uint8 (8-bit unsigned integer)

สรุป

cv2.imread() → NumPy array
shape = (สูง, กว้าง, 3)
สี = BGR
dtype = uint8

รายละเอียดรูปแบบข้อมูล

- รูปแบบ (shape) เป็น (สูง, กว้าง, จำนวนช่องสี)

`dims = img.shape`

↓
(Height, Width, Color channel)

(342, 548, 3)

ตัวแปร[0]
dims[0]

ตัวแปร[1]
dims[1]

ตัวแปร[2]
dims[2]

- รูปแบบสีเป็น BGR
(น้ำเงิน, เขียว, แดง) — ไม่ใช่ RGB

ช่องสีน้ำเงิน



B

0 - 255

ช่องสีเขียว



G

0 - 255

ช่องสีแดง



R

0 - 255

แนะนำชุดคำสั่ง OpenCV

วิธีการแสดงรูปภาพ



คำสั่ง `plt.imshow(...)` แสดงสีของรูปภาพเพี้ยน เนื่องจากข้อมูลรูปภาพที่อ่านเข้ามาผ่านคำสั่ง `cv2.imread(...)` มีรูปแบบการจัดเก็บข้อมูลแบบ BGR ขณะที่ คำสั่ง `plt.imshow(...)` มีการแสดงผลแบบ RGB ทำให้ผลลัพธ์ที่แสดงมีรูปแบบสีที่ผิดเพี้ยนไปจากที่ควรจะเป็น ดังนั้น จึงจำเป็นต้องทำการจัดเรียงองค์ประกอบของระนาบสีใหม่

คำสั่ง

```
output = cv2.cvtColor(input, cv2.COLOR_BGR2RGB)
```




แนะนำชุดคำสั่ง OpenCV

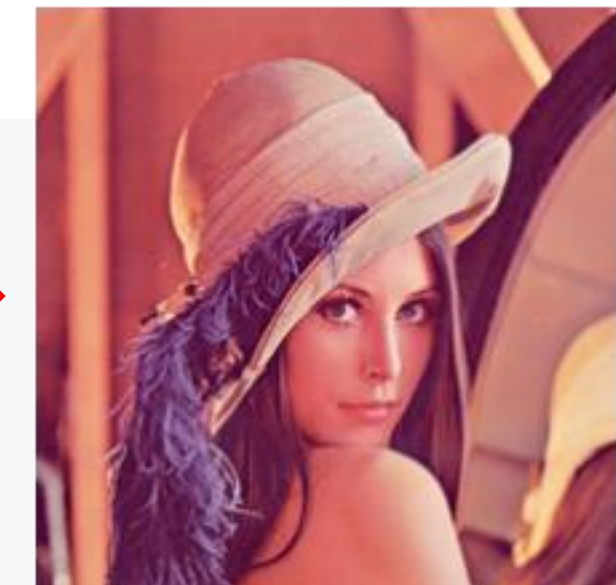
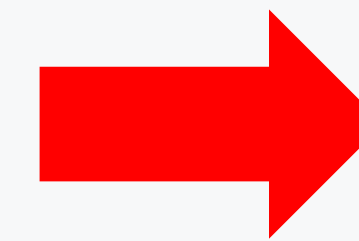
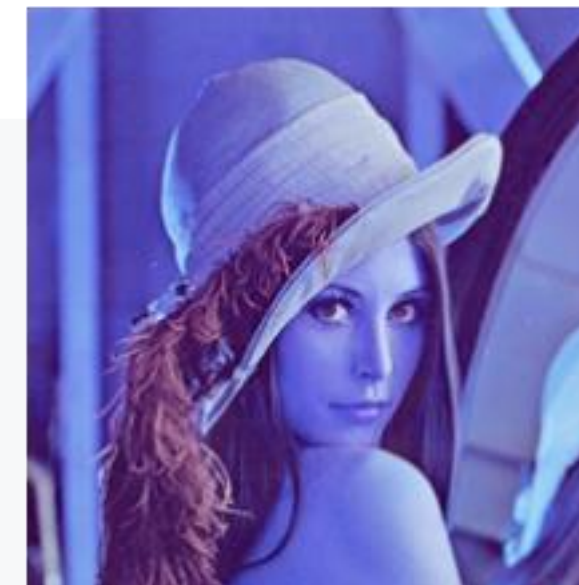
วิธีการแสดงรูปภาพ

ตัวอย่างเช่น

```
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
img_bgr = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)
img_rgb = cv2.cvtColor(img_bgr , cv2.COLOR_BGR2RGB)
```

```
plt.axis("off")
plt.imshow(img_rgb)
```





แนะนำชุดคำสั่ง OpenCV

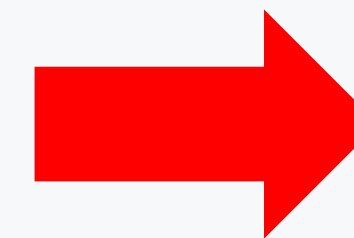
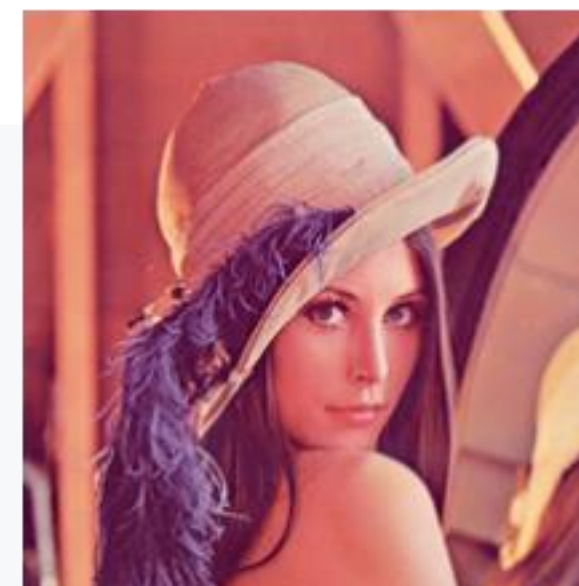
วิธีการแปลงปริภูมิสี (RGB -> YCbCr)

ตัวอย่างเช่น

```
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
img_bgr = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)
img_YCbCr = cv2.cvtColor(img_bgr , cv2.COLOR_BGR2YCrCb)
```

```
plt.axis("off")
plt.imshow(img_YCbCr)
```





แนะนำชุดคำสั่ง OpenCV

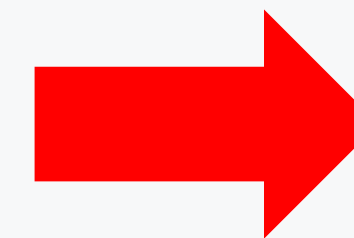
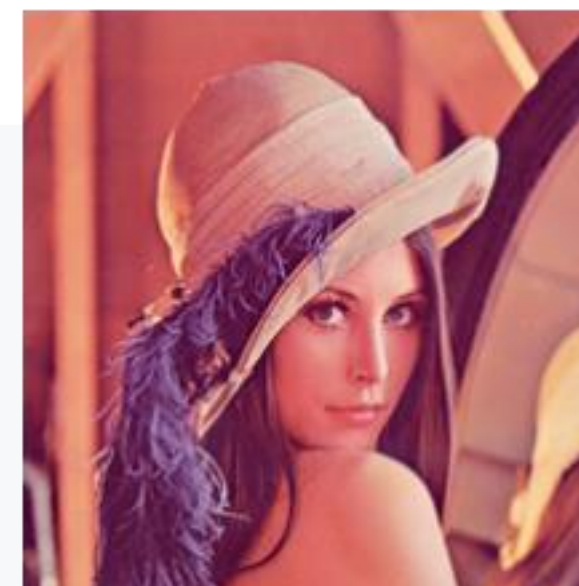
วิธีการแปลงปริภูมิสี (RGB -> HSV)

ตัวอย่างเช่น

```
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
img_bgr = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)
img_hsv = cv2.cvtColor(img_bgr , cv2.COLOR_BGR2HSV)
```

```
plt.axis("off")
plt.imshow(img_hsv)
```





แนะนำชุดคำสั่ง OpenCV

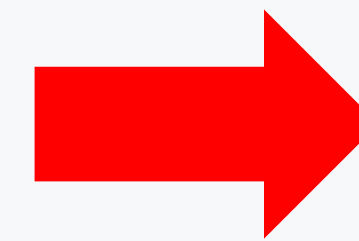
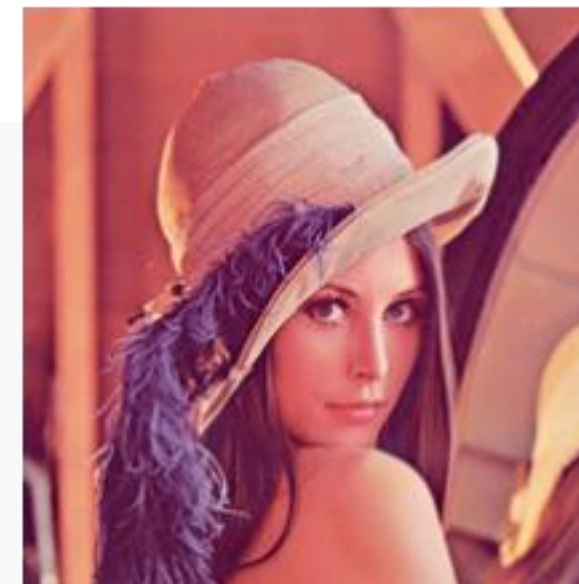
วิธีการแปลงปริภูมิสี (RGB -> Grayscale)

ตัวอย่างเช่น

```
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
img_bgr = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)
img_gray = cv2.cvtColor(img_bgr , cv2.COLOR_BGR2GRAY)
```

```
plt.axis("off")
plt.imshow(img_gray, cmap = 'gray', vmin = 0, vmax = 255, interpolation = 'none')
```





แนะนำชุดคำสั่ง OpenCV

วิธีการแปลงปริภูมิสี (RGB -> Binary)

ตัวอย่างเช่น

```
img_bgr = cv2.imread('/content/drive/My Drive/img/lenna_color.png',1)
out = cv2.cvtColor(img_bgr , cv2.COLOR_BGR2GRAY)

T = 127

# fix thresholding
temp0, bw0 = cv2.threshold(out,T,255,cv2.THRESH_BINARY)
# adaptive global
temp1, bw1 = cv2.threshold(out,T,255,cv2.ADAPTIVE_THRESH_MEAN_C)
# Otsu's
temp2, bw2 = cv2.threshold(out,T,255,cv2.THRESH_OTSU)
```



แนะนำชุดคำสั่ง OpenCV

วิธีการแปลงปริภูมิสี (RGB -> Binary)

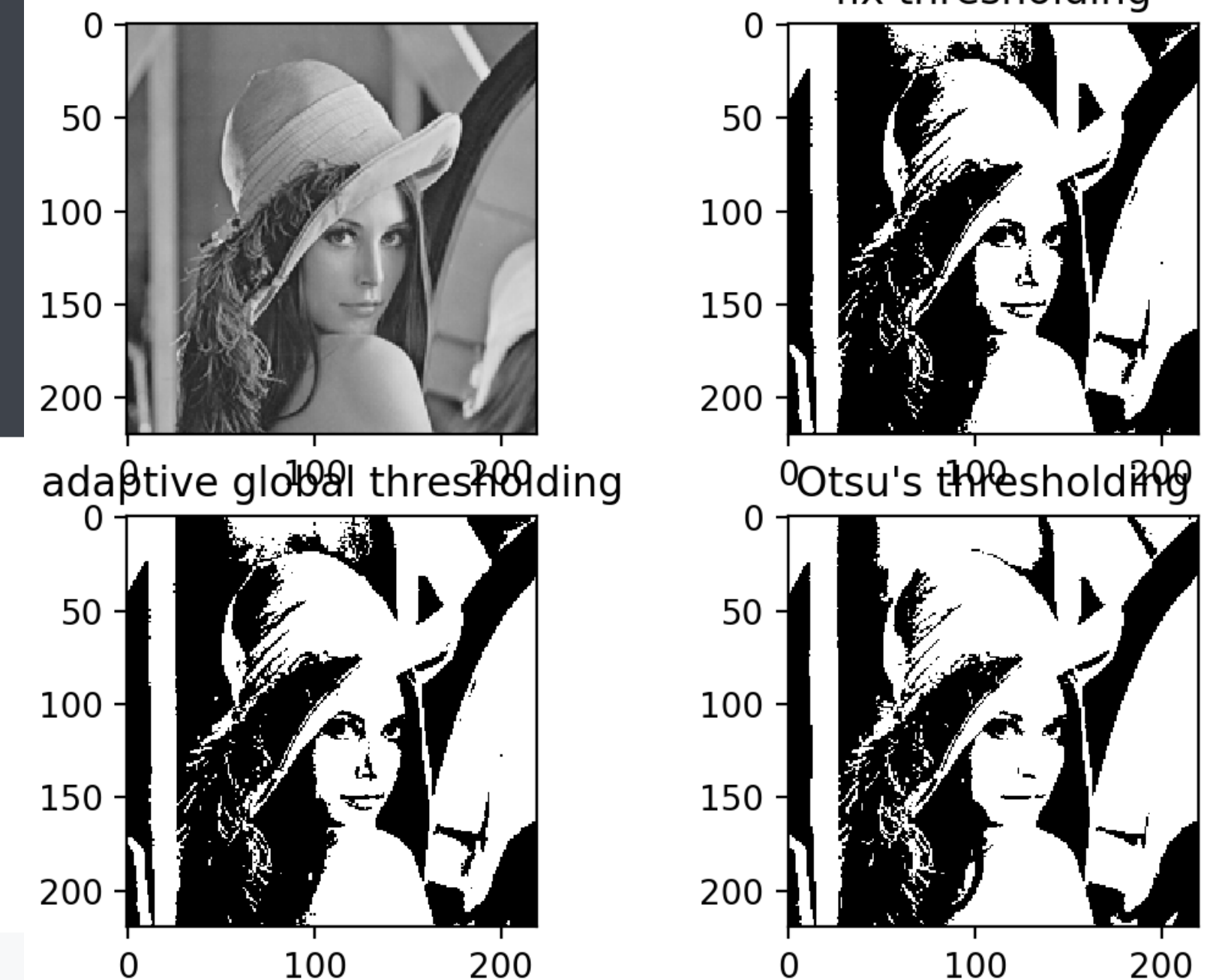
ตัวอย่างเช่น

```
plt.figure(), plt.subplot(2,2,1),  
plt.imshow(out, cmap='gray', vmin = 0, vmax = 255, interpolation='none')
```

```
plt.subplot(2,2,2),  
plt.imshow(bw0, cmap='gray', vmin = 0, vmax = 255, interpolation='none')
```

```
plt.subplot(2,2,3),  
plt.imshow(bw1, cmap='gray', vmin = 0, vmax = 255, interpolation='none')
```

```
plt.subplot(2,2,4),  
plt.imshow(bw2, cmap='gray', vmin = 0, vmax = 255, interpolation='none')
```





แนะนำชุดคำสั่ง OpenCV

ความสูง ความกว้าง และระนาบสีของรูปภาพ

นักศึกษาสามารถเรียกดูค่าความสูง ความกว้าง และระนาบสีของรูปภาพได้ โดยอาศัยคีย์ “.shape” ดังตัวอย่างต่อไปนี้

คำสั่ง

```
import cv2
from google.colab import drive
drive.mount('/content/drive')

im = cv2.imread('/content/.../img/lenna_color.png', 1)

dims = im.shape
```



แนะนำชุดคำสั่ง OpenCV

ความสูง ความกว้าง และระนาบสีของรูปภาพ

คำสั่ง

```
import cv2
from google.colab import drive
drive.mount('/content/drive')

im = cv2.imread('/content/.../img/lenna_color.png', 0)

height = im.shape[0]
width = im.shape[1]
# ถ้าใส่จะ Error เพราะข้อมูล Gray Scale มีแค่ 1 ระนาบ
channels = im.shape[2]
```

เพื่อระบุว่าเป็นภาพระดับสีเทา
(grayscale image)



แนะนำชุดคำสั่ง OpenCV

ความสูง ความกว้าง และระนาบสีของรูปภาพ

```
cv2.imread('lenna_gray.jpg')
```



ข้อมูลทั้ง 3 ระนาบมีค่าเหมือนกันทั้งหมด แต่กลับมองว่าเป็นข้อมูล 3 มิติ (h, w, d)

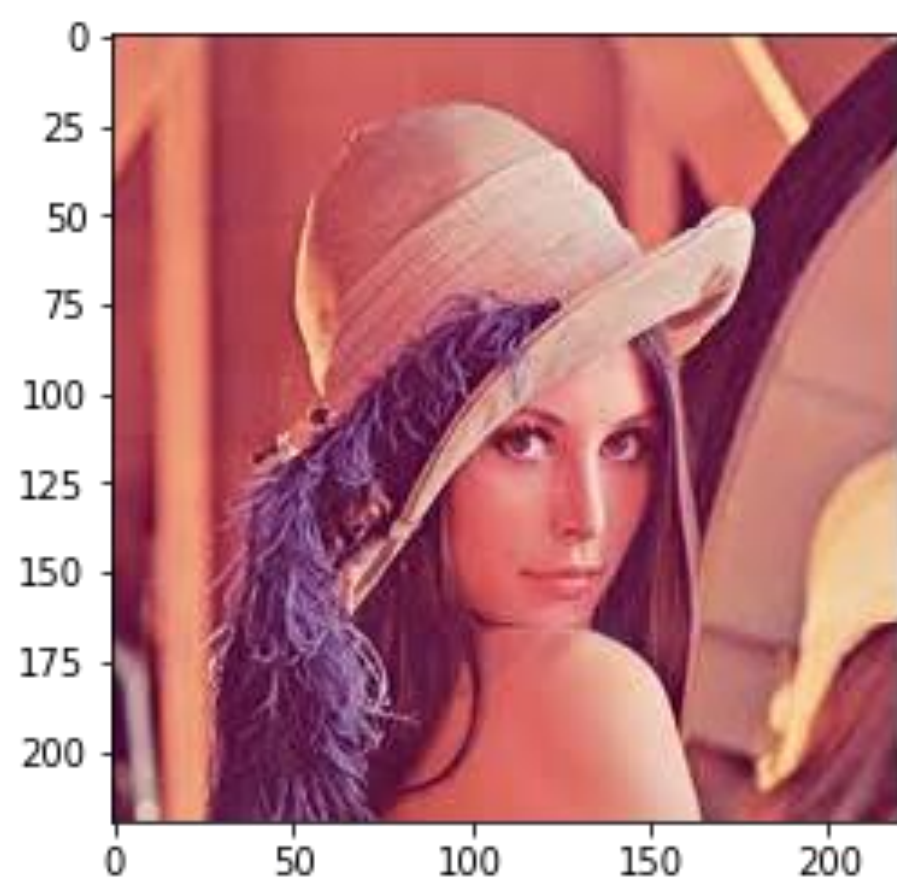
```
cv2.imread('lenna_gray.jpg', 0)
```



ข้อมูลมี 1 ระนาบเท่านั้น ทำให้สามารถมองว่าเป็นข้อมูล 2 มิติ (h, w)



คำสั่งแสดงขนาดของรูปภาพ



dimensions = (256, 256, 3)

ตัวแปร = ตัวแปรที่เก็บรูปภาพ.size

Img_size = im.size



Height x Width x Color channel



196,608 = 256 x 256 x 3



การเปลี่ยนแปลงขนาดของรูปภาพ

```
cv2.resize(image, dsize , interpolation)
```

Argument	Description
image	รูปภาพ
dsize	ขนาดของรูปภาพ (width, height)
interpolation	วิธีการสำหรับย่อ/ขยาย ขนาด INTER_LINEAR (Default) INTER_NEAREST INTER_AREA INTER_CUBIC INTER_LANCZOS4



การเปลี่ยนแปลงขนาดของรูปภาพ

ให้นักศึกษาพิจารณาตัวอย่างต่อไปนี้

```
import cv2

im = cv2.imread('lenna.jpg', 0)
h = 128
w = 128

reImg = cv2.resize(im, (h, w), interpolation = cv2.INTER_CUBIC)

print(im.shape)
print(reImg.shape)
```




ประเภทของค่าสีของรูปภาพ



โดยทั่วไปแล้วค่าความเข้มแสงของแต่ละพิกเซลจะมี 2 ประเภท (ไม่ว่าจะเป็นภาพสีหรือภาพระดับสีเทา) ได้แก่

- 0 – 1 ใน OpenCV จะเรียกว่าประเภท double
- 0 – 255 ใน OpenCV จะเรียกว่าประเภท uint8

ซึ่งนักศึกษาสามารถตรวจสอบได้จากคีย์ “. dtype” ดังตัวอย่างต่อไปนี้

```
import cv2
```

```
img = cv2.imread('lenna.jpg',0)  
print('Type : ', img.dtype)
```



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

การอ้างอิงค่าความเข้มแสงของแต่ละพิกเซลจะอาศัยวงเล็บก้ามปู [...] โดยระบุตำแหน่งเป็นระนาบ [row, col] สำหรับข้อมูล 2 มิติ ตัวอย่างเช่น

ตัวแปร = ตัวแปรที่เก็บรูปภาพ [row, col]

ขณะที่ [row, col, dim] สำหรับข้อมูล 3 มิติเท่านั้น ตัวอย่างเช่น

ตัวแปร = ตัวแปรที่เก็บรูปภาพ [row, col, dim]

เลขตำแหน่งอ้างอิงเริ่มจาก 0



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

สำหรับภาพระดับสีเทาจะแสดงเพียงค่าเดี่ยว เช่น

```
i = img[100,100]  
print( i )           # i = 100
```

สำหรับภาพสีจะแสดงค่า [B, G, R] เช่น [100, 20, 30] (ถ้าไม่มีการเรียงใหม่)

```
i = img[100,100]  
print( i )           # i = [100, 20, 30]
```




การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

สำหรับภาพสีจะแสดงค่า [B, G, R] เช่น [100, 20, 30] (ถ้าไม่มีการเรียงใหม่)

```
i = img[100, 100, 2]
print( i )                                # i = 30
```

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import cv2
img = cv2.imread('lenna.png', 0) # ภาพสีเทา

print(img.dtype)
print(img[1,1])
print(img[1,1,0])
```

```
uint8
167
-----
IndexError                                Traceback (most recent call last)
<ipython-input-3-38ca911e3a9c> in <module>
      2 print(img.dtype)
      3 print(img[1,1])
----> 4 print(img[1,1,0])

IndexError: too many indices for array: array is 2-dimensional, but 3 were indexed
```



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

ตัวอย่าง

```
import cv2
img = cv2.imread('lenna.png', 1)    # ภาพสี

print(img[100,100])                  # [100, 20, 30]

print(img[100,100,0])                 # 100
print(img[100,100,1])                 # 20
print(img[100,100,2])                 # 30
```



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

การเลือกค่าความเข้มแสงของพิกเซล คือ “การอ้างอิงตำแหน่ง” ซึ่งนักศึกษสามารถมองว่าตำแหน่งที่อ้างอิง (`img[r, c]`) เป็นตัวแปรธรรมดาและสามารถนำไปใช้งานต่าง ๆ ได้ เช่น

```
import cv2
img = cv2.imread('lenna.png', 1)    # ภาพสี

print(img[100, 100])                 # [100, 20, 30]

img[100, 100, 0] = 2;
print(img[100, 100, 0])               # 2

img[100, 100, 0] = img[100, 100, 0] + 11;
print(img[100, 100, 0])               # 33
```




การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

สำหรับภาพระดับสีเทา

ตัวแปรที่เก็บรูปภาพ[row, col] = ค่าระดับสีเทา

`img[100,100] = 255`

สำหรับภาพสีจะแสดงค่า [B, G, R] (ถ้าไม่มีการเรียงใหม่)

ตัวแปรที่เก็บรูปภาพ[row, col] = [B, G, R]

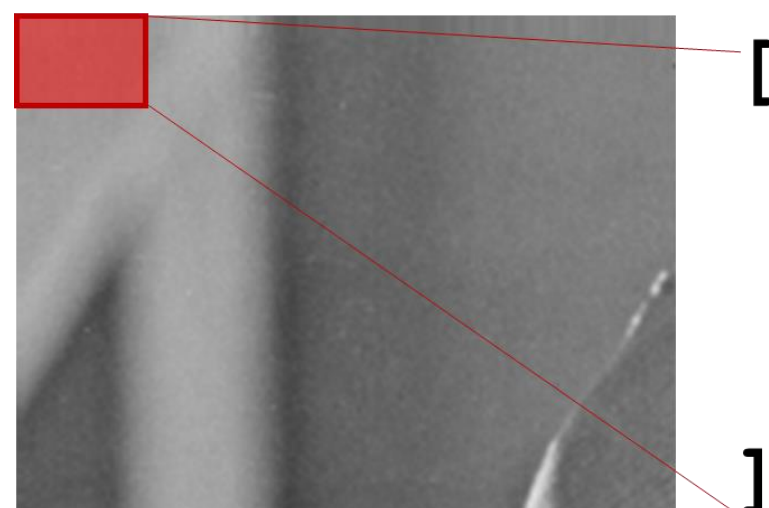
`img[100,100] = [255,10,20]`

ตัวแปรที่เก็บรูปภาพ[row, col, color channel] = ค่าสี

`img[100,100,0] = 255`



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)



[
]
]

```
[161 161 157 161 161]  
[161 161 157 161 161]  
[163 155 159 159 159]
```

ถ้านักศึกษาต้องการเลือกข้อมูลเป็นช่วงสามารถอาศัย
ตัวดำเนินการ colon (:) ร่วมกับวงเล็บปีกกาได้ เช่น

สำหรับภาพระดับสีเทา : `[row, column]`

ตัวแปร `[start:stop-1:step, start:stop-1:step]`

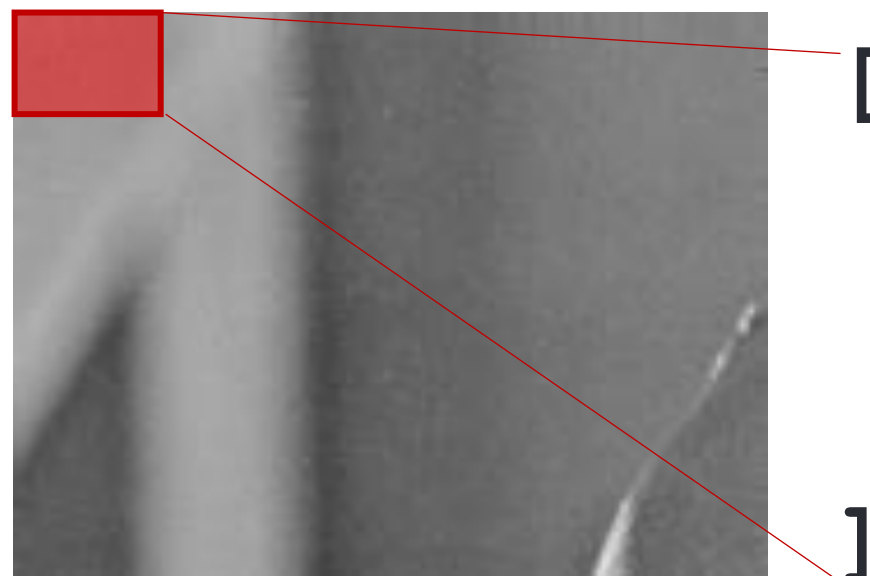
สำหรับภาพสีจะแสดงค่า `[B, G, R]` (ถ้าไม่มีการเรียงใหม่) : `[row, column, dim]`

ตัวแปร `[start:stop-1:step, start:stop-1:step, start:stop-1:step]`



การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

กำหนดให้

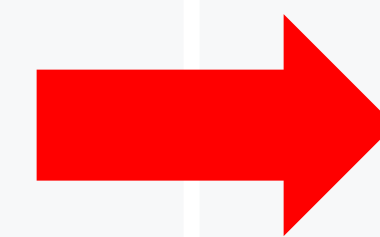


```
[  
  [168 167 167 170 167]  
  [168 167 167 170 167]  
  [169 167 167 167 167]  
]
```

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import cv2  
  
img = cv2.imread('lenna.png', 0)  
  
print('Ex 1 : \n ', img[0, 0:5])  
print('Ex 2 : \n ', img[0:3, 0:5])  
print('Ex 3 : \n ', img[0:3:2, 0:5:2])
```

ผลลัพธ์



```
Ex 1 :  
  [168 167 167 170 167]  
Ex 2 :  
  [[168 167 167 170 167]  
   [168 167 167 170 167]  
   [169 167 167 167 167]]  
Ex 3 :  
  [[168 167 167]  
   [169 167 167]]
```




การเลือกค่าความเข้มแสงของพิกเซลในภาพ (เดี่ยว)

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

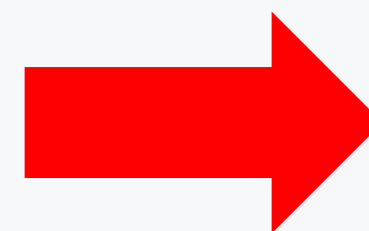
```
import cv2
img = cv2.imread('lenna.png', 0)

print('Ex 1 : \n ', img[0:3,0:3])
img[0:3,0:3] = 99;

print('Ex 2 : \n ', img[0:3,0:3])
img[0:3,0:3] = [[11, 12, 13],
[14, 15, 16], [17, 18, 19]]

print('Ex 3 : \n ', img[0:3,0:3])
print('Ex 4 : \n ', img[:,:]) >> ??
```

ผลลัพธ์



```
Ex 1 :
[[161 161 157]
 [161 161 157]
 [163 155 159]]
Ex 2 :
[[99 99 99]
 [99 99 99]
 [99 99 99]]
Ex 3 :
[[11 12 13]
 [14 15 16]
 [17 18 19]]
```



การบันทึกข้อมูลภาพลงบนเครื่องคอมพิวเตอร์

นักศึกษาสามารถบันทึกตัวแปรที่เก็บรูปภาพลงบนเครื่องคอมพิวเตอร์ได้โดยอาศัยคำสั่ง

```
ตัวแปร = cv2.imwrite(ที่อยู่ของรูปภาพ/ชื่อรูปภาพ.นามสกุล',ตัวแปรที่เก็บรูปภาพ)
```

โดยที่ ตัวแปร เป็น True เมื่อสามารถบันทึกข้อมูลได้สำเร็จ และมีค่าเป็น False เมื่อไม่สามารถบันทึกข้อมูลได้

ตัวอย่างเช่น

```
import cv2
```

```
img = cv2.imread('lena.png', 0)
```

```
status = cv2.imwrite('lena2.png', img)
```



หัวข้อ

- แนะนำชุดคำสั่ง OpenCV
- แนะนำ อาร์เรย์ (Array) ในชุดคำสั่ง numpy
 - วิธีการสร้าง
 - วิธีการอ้างอิงตำแหน่ง
 - ตัวดำเนินการทางคณิตศาสตร์
 - การเชื่อมต่ออาร์เรย์
 - การปรับขนาดและคัดลอก
- แนะนำชุดคำสั่ง matplotlib



แนะนำไลบรารี numpy

numpy เป็นชุดคำสั่งสำหรับการคำนวณทางตัวเลข ที่สร้างขึ้นจากภาษาไพธอน เมื่อติดตั้ง Anaconda แล้วจะมี numpy มาด้วย โดยมีฟังก์ชันที่สำคัญ ได้แก่ อาร์เรย์ที่เก็บ ชุดตัวเลข หรือ เมตริก ที่นำไปแทนพิกเซล (pixel) ของรูปภาพ การนำเข้า numpy ดังนี้

```
import numpy as np
```




อาร์เรย์ (Array)

อาร์เรย์ของ numpy นำมาใช้ในการคำนวณจะมีประสิทธิภาพมากกว่า List ของไพธอน โดยเฉพาะข้อมูลที่มีขนาดใหญ่ ข้อมูลที่มีหลายมิติ จะแปลงเป็นอาร์เรย์ของ NumPy เช่นการเก็บข้อมูลภาพ การสร้างอาร์เรย์มีหลายวิธี แต่วิธีพื้นฐานสุด คือ สร้างขึ้นมาจากลิสต์, ทูเพิล หรือเรนจ์ โดยใช้คำสั่ง

```
numpy.array (ข้อมูล , ประเภทข้อมูล)
```



อาร์เรย์ (Array)

- ตัวอย่าง การสร้างอาร์เรย์ 1 มิติ

```
import numpy as np
ari = np.array([1,2,3,4])
ari = np.array([1,2,3,4], 'int')
ari = np.array([1,2,3,4], dtype='int64')
ari = np.array((1,2,3,4))
```

- ตัวอย่าง การสร้างอาร์เรย์ 2 มิติ

```
import numpy as np
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
print(arr)
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```



อาร์เรย์ (Array)

- ฟังก์ชันสำหรับสร้างอาร์เรย์ 1 มิติ

คำสั่ง	คำอธิบาย
<code>arange(start, stop-1, step)</code>	สร้างสมาชิกตัวเลขในรูปแบบอาร์เรย์ที่อยู่ในช่วง start ถึง stop
<code>linspace(start, stop, size)</code>	สร้างสมาชิกตัวเลขจำนวน Size ตัวที่อยู่ในช่วง start ถึง stop ในรูปแบบอาร์เรย์

- ฟังก์ชันสำหรับสร้างอาร์เรย์ 2 มิติ

คำสั่ง	คำอธิบาย
<code>array([seq])</code>	สร้างอาร์เรย์โดยระบุค่าสมาชิกที่มีอยู่ใน 2D List หรือ Tuple
<code>zeros([rows, cols])</code>	สร้างอาร์เรย์ที่สมาชิกทุกตัวมีค่าเป็น 0 โดยระบุจำนวนแถวและหลัก
<code>ones([rows, cols])</code>	สร้างอาร์เรย์ที่สมาชิกทุกตัวมีค่าเป็น 1 โดยระบุจำนวนแถวและหลัก



อาร์เรย์ (Array)

- ตัวอย่างที่ 1

```
import numpy as np
arr1 = np.arange(1,10,2)
arr2 = np.linspace(0,10, 4)
print(arr1)
print(arr2)
```

ผลลัพธ์

```
[1 3 5 7 9]
[ 0.          3.33333333  6.66666667 10.]
```

- ตัวอย่างที่ 2

```
import numpy as np
arr1 = np.array([[0,1],[2, 9]])
arr2 = np.zeros([2,3])
arr3 = np.ones([3,2])
print(arr1)
print(arr2)
print(arr3)
```

ผลลัพธ์

```
[[0 1]
 [2 9]]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```




อาร์เรย์ (Array)

การอ้างอิงตำแหน่งสมาชิกในอาร์เรย์มีด้วยกัน 2 ระบบ ได้แก่ แบบต่อเนื่อง (Index Slicing) และ แบบตามใจ (Fancy Indexing)

- **แบบต่อเนื่อง** การอ้างอิงแบบนี้อาศัยเครื่องหมายปีกกา [] โดยภายในวงเล็บให้ระบุเป็นตำแหน่ง **start : stop-1: step**
- **แบบตามใจ** การอ้างอิงแบบนี้อาศัยเครื่องหมายปีกกา [] เช่นเดียวกัน แต่ภายในวงเล็บให้ระบุ**เป็นตำแหน่งเป็นตัวเลข**ได้เลย



อาร์เรย์ - อ้างอิงแบบต่อเนื่อง (Index Slicing)

- ตัวอย่าง การอ้างอิงตำแหน่งสมาชิกในอาร์เรย์ 1 มิติ

```
import numpy as np
ari = np.array([1,2,3,4])
print(ari)                # [1 2 3 4]
print(ari[1])             # 2
print(ari[1:3])           # [2 3]
```

- ตัวอย่าง การอ้างอิงตำแหน่งสมาชิกในอาร์เรย์ 2 มิติ

```
import numpy as np
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
print(arr[1,1])           # 4
print(arr[:,1])           # [2, 4, 6, 8]
print(arr[[0, 3],:])      # [[1, 2], [7, 8]]
```



อาร์เรย์ – อ้างอิงแบบตามใจ (Fancy Indexing)

- ตัวอย่าง การอ้างอิงตำแหน่งสมาชิกในอาร์เรย์ 1 มิติ

```
import numpy as np
ari = np.array([21, 12, 43, 4])
print(ari)
print(ari[[2, 3, 0]])
print(ari[[2, 1]])
```

ผลลัพธ์

```
[21 12 43  4]
[43  4 21]
[43 12]
```

- ตัวอย่าง การอ้างอิงตำแหน่งสมาชิกในอาร์เรย์ 2 มิติ

```
import numpy as np
arr = np.array([[21, 11, 2], [13, 0, 24], [51, 4, 36], [0, 17, 38]])
print(arr[[2, 3, 1], 1])
print(arr[:, [2, 0]])
print(arr[[0, 3], :])
```

ผลลัพธ์

```
[ 4 17  0]
[[ 2 21]
 [24 13]
 [36 51]
 [38  0]]
[[21 11  2]
 [ 0 17 38]]
```




อาร์เรย์ (Array)

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np

print('Hello')
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
print(arr)
arr[[0, 3],:] = 9;
print(arr)
i = arr[[1,2], :]
print(i)
```

ผลลัพธ์

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[9 9]
 [3 4]
 [5 6]
 [9 9]]
[[3 4]
 [5 6]]
```




อาร์เรย์ (Array) - แบบฝึกหัด

จงตอบคำถามต่อไปนี้ โดยกำหนดให้

$x = [[0, 7, 5, 9], [2, 1, 9, 8], [8, 2, 3, 4], [1, 2, 0, 3], [4, 6, 7, 9]]$

- $x[1:2, 0:4]$
- $x[1:3:2, 0:3:2]$
- $x[3, 0:3:2]$
- $x[1:3, :]$
- $x[0:, 0:1]$
- $x[0::2, 0::2]$
- $x[[0, 2, 4], :]$
- $x[[2, 1, 3], [0, 1, 3]]$



อาร์เรย์ (Array) - แบบฝึกหัด

$$x = \begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix}$$

1. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$

2. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$

3. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$

4. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$

5. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$

6. $\begin{bmatrix} 0 & 7 & 5 & 9 \\ 2 & 1 & 9 & 8 \\ 8 & 2 & 3 & 4 \\ 1 & 2 & 0 & 3 \\ 4 & 6 & 7 & 9 \end{bmatrix} \gg x[_, _]$



ตัวดำเนินการทางคณิตศาสตร์

ตัวดำเนินการ	ความหมาย
+ (บวก)	รองรับทั้งสเกลาร์และเมตริกซ์ แต่ถ้าเมตริกซ์ต้องมีขนาดเท่ากัน
- (ลบ)	รองรับทั้งสเกลาร์และเมตริกซ์ แต่ถ้าเมตริกซ์ต้องมีขนาดเท่ากัน
* (คูณ)	รองรับทั้งสเกลาร์และเมตริกซ์ แต่ถ้าเมตริกซ์ต้องมีขนาดเท่ากัน
np.dot(x, y) (Dot product)	รองรับแต่เมตริกซ์
/ (หาร)	รองรับทั้งสเกลาร์และเมตริกซ์ แต่ถ้าเมตริกซ์ต้องมีขนาดเท่ากัน
** (ยกกำลัง)	รองรับแต่สเกลาร์
np.transpose(y) (Transpose)	รองรับแต่เมตริกซ์



อาร์เรย์ – ตัวดำเนินการทางคณิตศาสตร์แบบจุด

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np
```

```
x = np.array([[1, 2, 3], [ 4, 5, 6]])
```

```
y = np.array([[ -1, 1, 5], [ 2, 3, 1]])
```

```
print(x + y)
```

```
print(x - y)
```

```
print(x * y)
```

```
print(x / y)
```

```
print(x ** 2)
```

ผลลัพธ์

```
[[0 3 8]
 [6 8 7]]
[[ 2  1 -2]
 [ 2  2  5]]
[[-1  2 15]
 [ 8 15  6]]
[[-1.         2.         0.6        ]
 [ 2.         1.66666667  6.        ]]
[[ 1  4  9]
 [16 25 36]]
```




อาร์เรย์ – ตัวดำเนินการทางคณิตศาสตร์แบบเมตริกซ์

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np

x = np.array([3, 4, 2])
y = np.array([[13, 9, 7, 15],
               [8, 7, 4, 6],
               [6, 4, 0, 3]])

print(' ans1 : ', np.dot(x, y))
```

ผลลัพธ์

```
ans1 : [83 63 37 75]
```



อาร์เรย์ – ตัวดำเนินการทางคณิตศาสตร์แบบเมตริกซ์

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np

x = np.array([[1, 2, 3], [ 4, 5, 6]])
y = np.array([[-1, 1, 5], [ 2, 3, 1]])

print(' ans1 : ', np.dot(x, np.transpose(y)))
```

ผลลัพธ์

```
ans1 :  [[16 11]
 [31 29]]
```



อาร์เรย์ – การเชื่อมต่อเมตริกซ์

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np

x = np.array([-3, 0, 2, 4, 6])
y = np.array([5, 15, 15, 20, 10])

z1 = np.concatenate([x, y])
z2 = np.concatenate([[x], [y]])

print('out 1 : ', z1)
print('out 2 : ', z2)
```

ผลลัพธ์

```
out 1 : [-3  0  2  4  6  5 15 15 20 10]
out 2 : [[-3  0  2  4  6]
 [ 5 15 15 20 10]]
```




อาร์เรย์ – การคัดลอกเมตริกซ์

ให้นักศึกษาพิจารณา 2 โปรแกรมต่อไปนี้

```
import numpy as np
```

```
x = np.array([-3, 0, 2, 4, 6])
```

```
y = x
```

```
print('x : ', x)
```

```
print(' ===== '
```

```
y[0] = 10
```

```
print('x : ', x)
```

```
print('y : ', y)
```

ผลลัพธ์

```
x : [-3  0  2  4  6]
=====
x : [10  0  2  4  6]
y : [10  0  2  4  6]
```

```
import numpy as np
```

```
x = np.array([-3, 0, 2, 4, 6])
```

```
y = x.copy()
```

```
print('x : ', x)
```

```
print(' ===== '
```

```
y[0] = 10
```

```
print('x : ', x)
```

```
print('y : ', y)
```

ผลลัพธ์

```
x : [-3  0  2  4  6]
=====
x : [-3  0  2  4  6]
y : [10  0  2  4  6]
```




อาร์เรย์ – การคัดลอกเมตริกซ์

ให้นักศึกษาพิจารณา 2 โปรแกรมต่อไปนี้

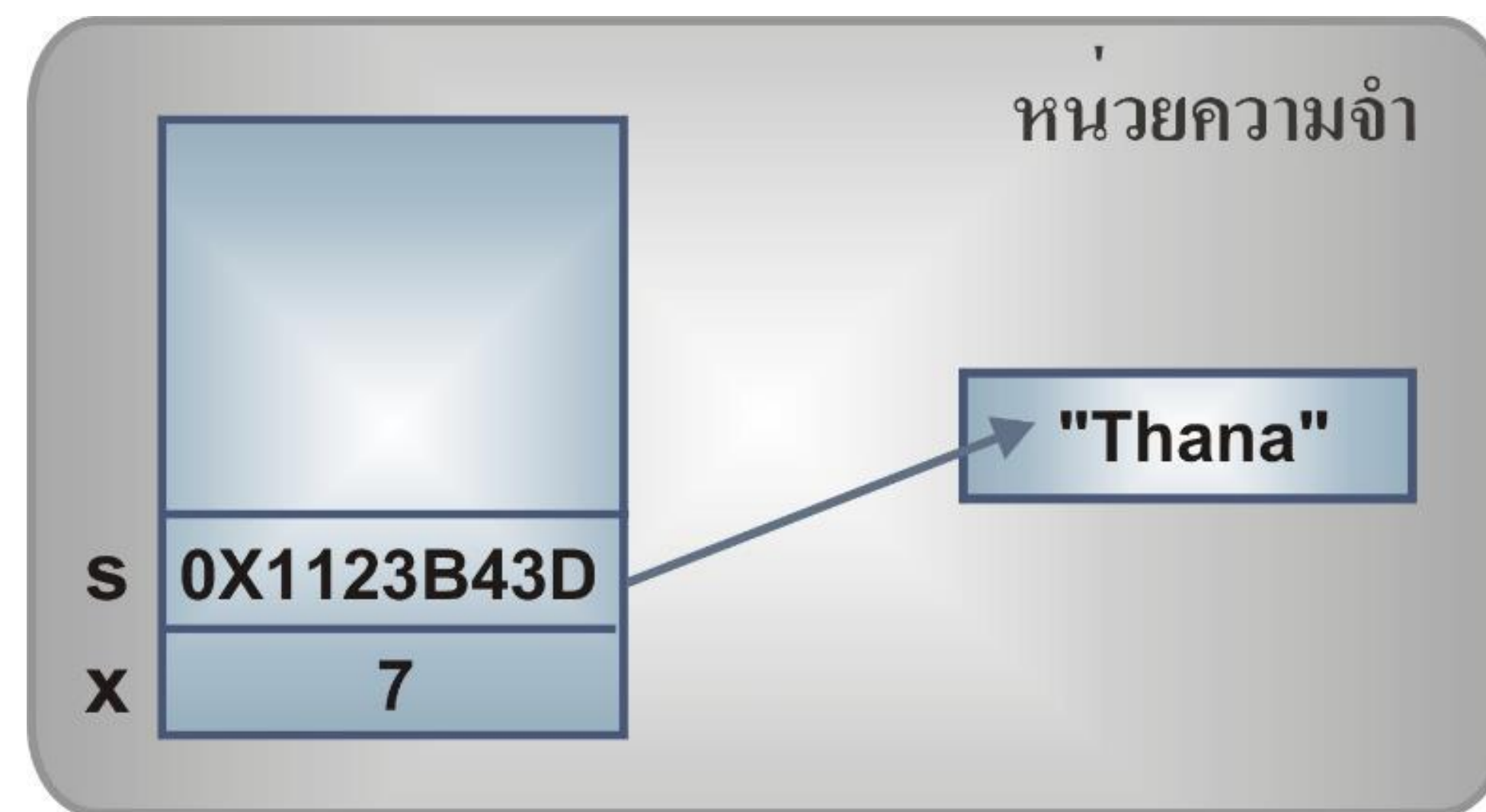
```
import numpy as np

x = np.array([-3, 0, 2, 4, 6])
y = x

print('x : ', x)
print(' ===== ')

y[0] = 10

print('x : ', x)
print('y : ', y)
```



ผลลัพธ์

```
x :  [-3  0  2  4  6]
=====
x :  [10  0  2  4  6]
y :  [10  0  2  4  6]
```



อาร์เรย์ – การปรับขนาด

ให้นักศึกษาพิจารณาโปรแกรมต่อไปนี้

```
import numpy as np

x = np.array([-3, 0, 2, 4, 6, 7])
y1 = np.resize(x, [2, 3])
y2 = np.resize(x, [3, 2])
y3 = np.resize(x, [6, 1])

print('x : ', x)
print('y1 : ', y1)
print('y2 : ', y2)
print('y3 : ', y3)
```

ผลลัพธ์

```
x : [-3  0  2  4  6  7]
y1 : [[-3  0  2]
      [ 4  6  7]]
y2 : [[-3  0]
      [ 2  4]
      [ 6  7]]
y3 : [[-3]
      [ 0]
      [ 2]
      [ 4]
      [ 6]
      [ 7]]
```



หัวข้อ

- แนะนำชุดคำสั่ง OpenCV
- แนะนำ อาร์เรย์ (Array) ในชุดคำสั่ง numpy
 - วิธีการสร้าง
 - วิธีการอ้างอิงตำแหน่ง
 - ตัวดำเนินการทางคณิตศาสตร์
 - การเชื่อมต่ออาร์เรย์
 - การปรับขนาดและคัดลอก
- แนะนำชุดคำสั่ง matplotlib

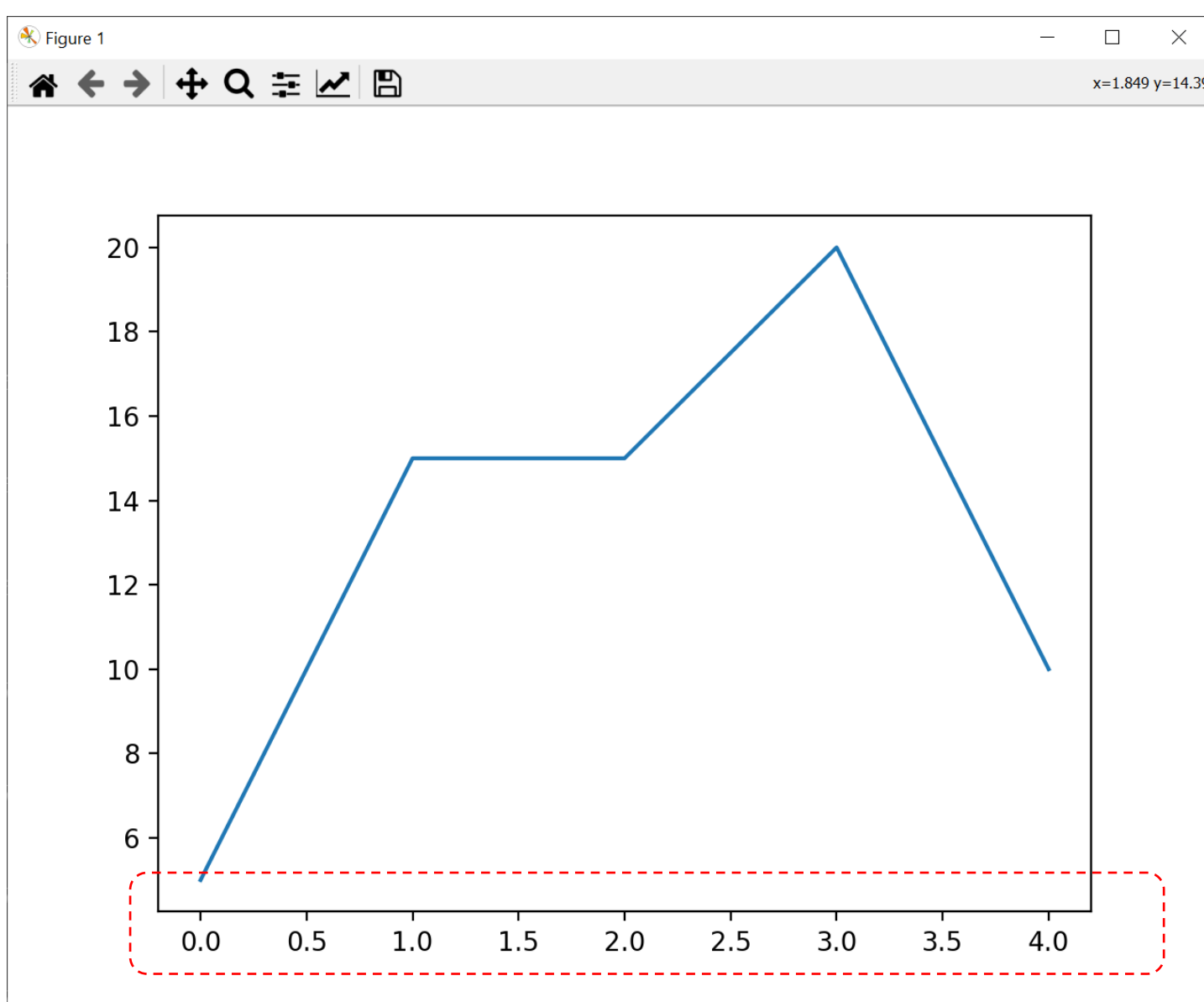


การวาดกราฟเส้น

กราฟเส้น (Linear Plot) เป็นการลากเส้นเชื่อมระหว่างจุดที่แสดงความสัมพันธ์ของข้อมูล โดยอาศัยคำสั่ง `plot` จากไลบรารี `matplotlib`

- รูปแบบที่ 1 เป็นเวกเตอร์ 1 ตัว

```
import matplotlib.pyplot as plt
x = [5, 15, 15, 20, 10]
plt.plot(x)
plt.show()
```

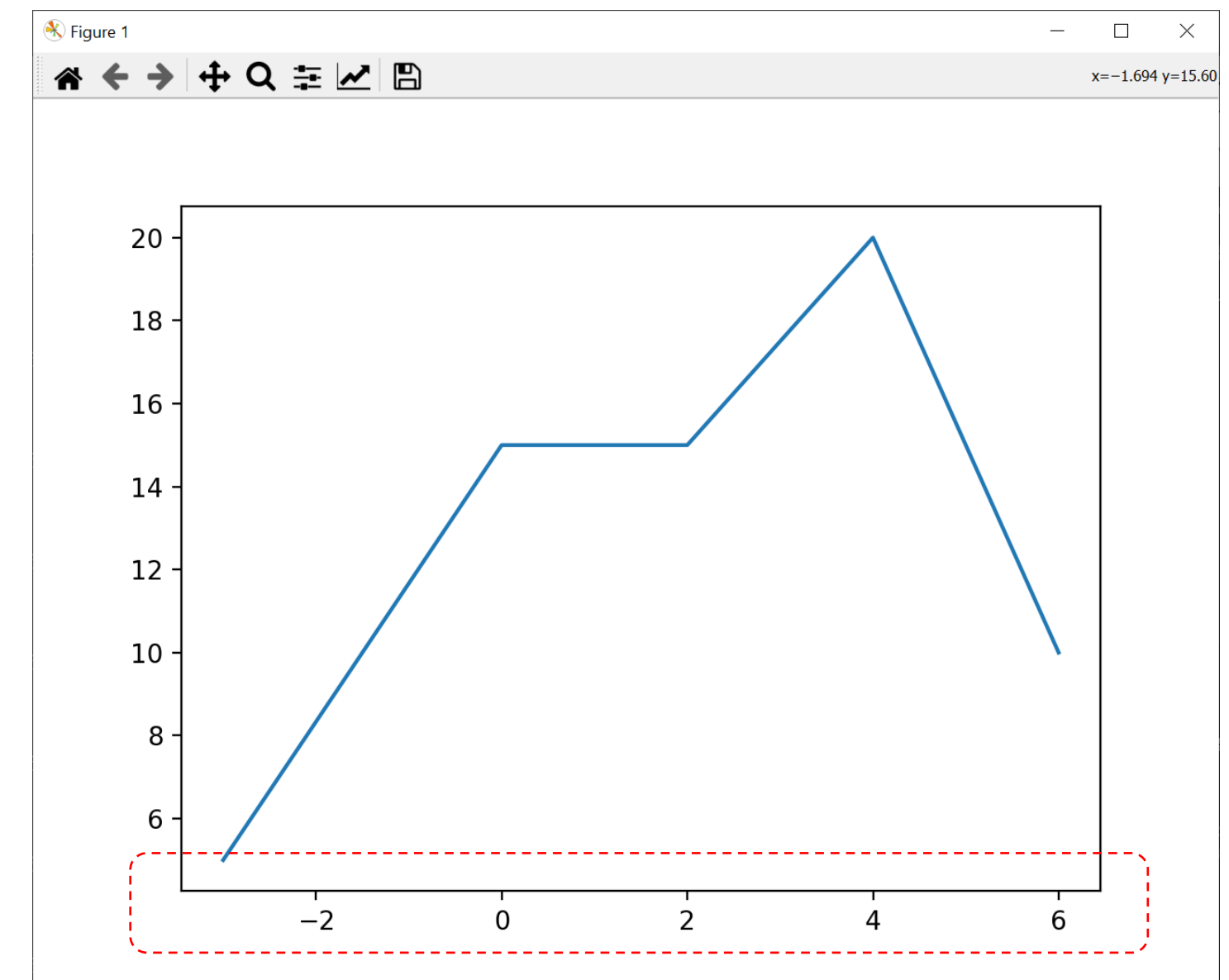




การวาดกราฟเส้น

- รูปแบบที่ 2 เป็นเวกเตอร์ 2 ตัว

```
import matplotlib.pyplot as plt
x = [-3, 0, 2, 4, 6]
y = [5, 15, 15, 20, 10]
plt.plot(x, y)
plt.show()
```





การวาดกราฟเส้น (2 เส้นทับกัน)

```
import matplotlib.pyplot as plt
```

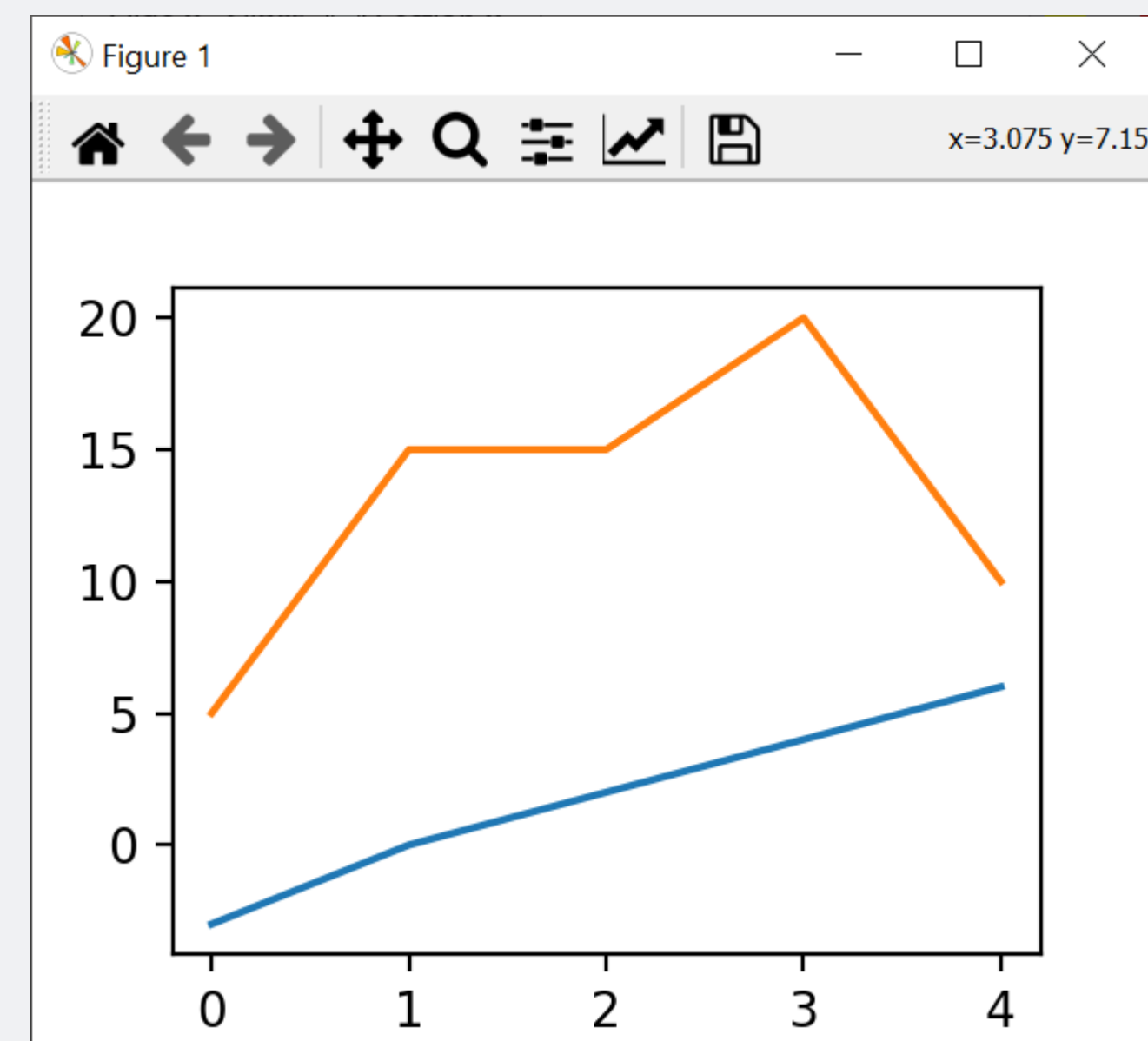
```
x = [-3, 0, 2, 4, 6]
```

```
y = [5, 15, 15, 20, 10]
```

```
plt.plot(x)
```

```
plt.plot(y)
```

```
plt.show()
```





การวาดกราฟแท่ง

กราฟแท่ง (Bar Plot) เป็นการวาดเป็นรูปแท่งสีเหลี่ยมนิยมใช้เพื่อเปรียบเทียบความแตกต่างระหว่างข้อมูล โดยอาศัยคำสั่ง `bar` จากไลบรารี `matplotlib`

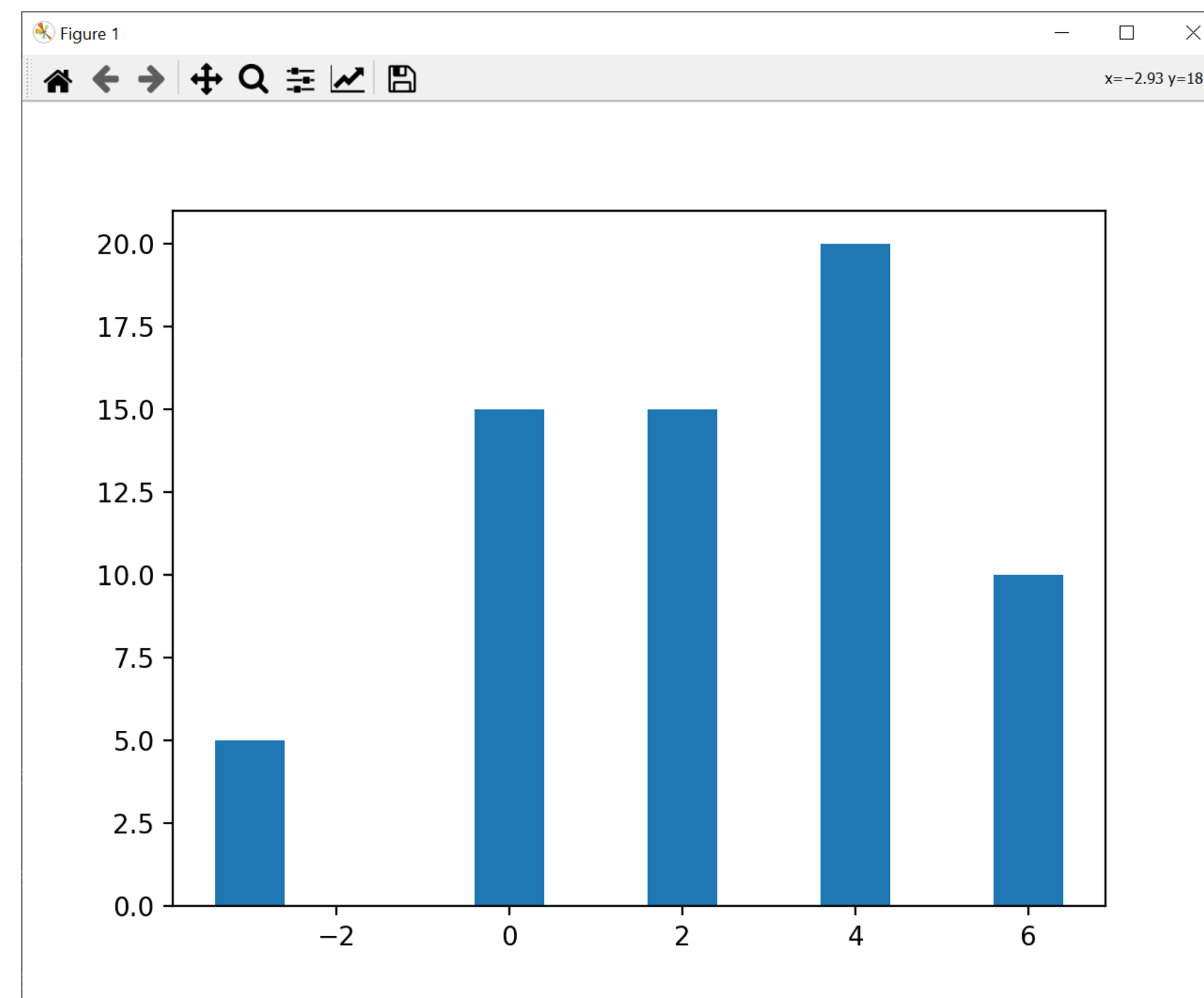
```
import matplotlib.pyplot as plt
```

```
x = [-3, 0, 2, 4, 6]
```

```
y = [5, 15, 15, 20, 10]
```

```
plt.bar(x, y)
```

```
plt.show()
```





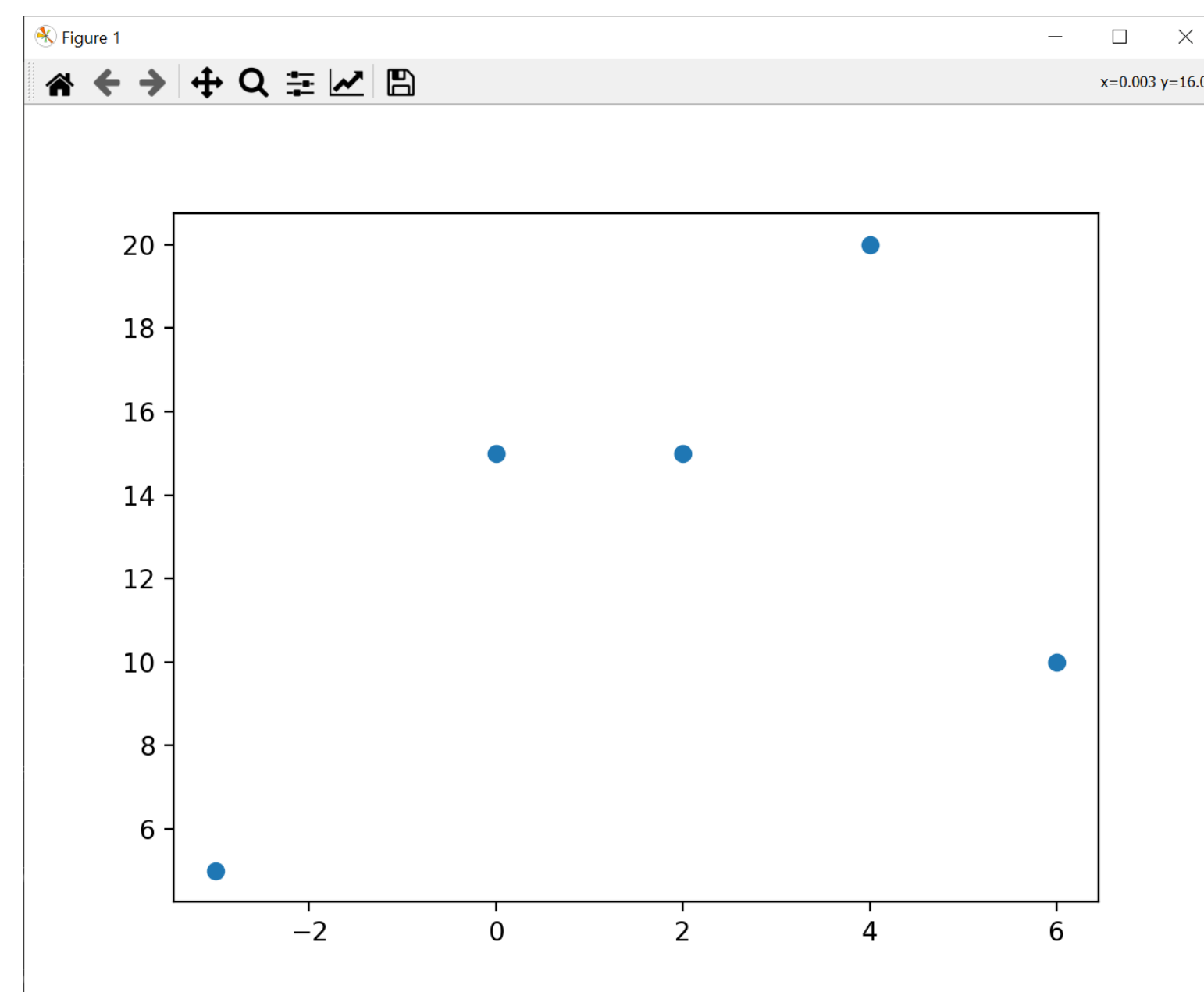
การวาดกราฟแบบกระจายกลุ่มข้อมูล

กราฟกระจาย (Scatter) เป็นการวาดการกระจายตัวข้อมูลใน 2 ตัวแปร นิยมใช้เปรียบเทียบ ตำแหน่ง ปริมาณ และการกระจายตัวของข้อมูล โดยอาศัยคำสั่ง scatter

```
import matplotlib.pyplot as plt

x = [-3, 0, 2, 4, 6]
y = [5, 15, 15, 20, 10]

plt.scatter(x, y)
plt.show()
```





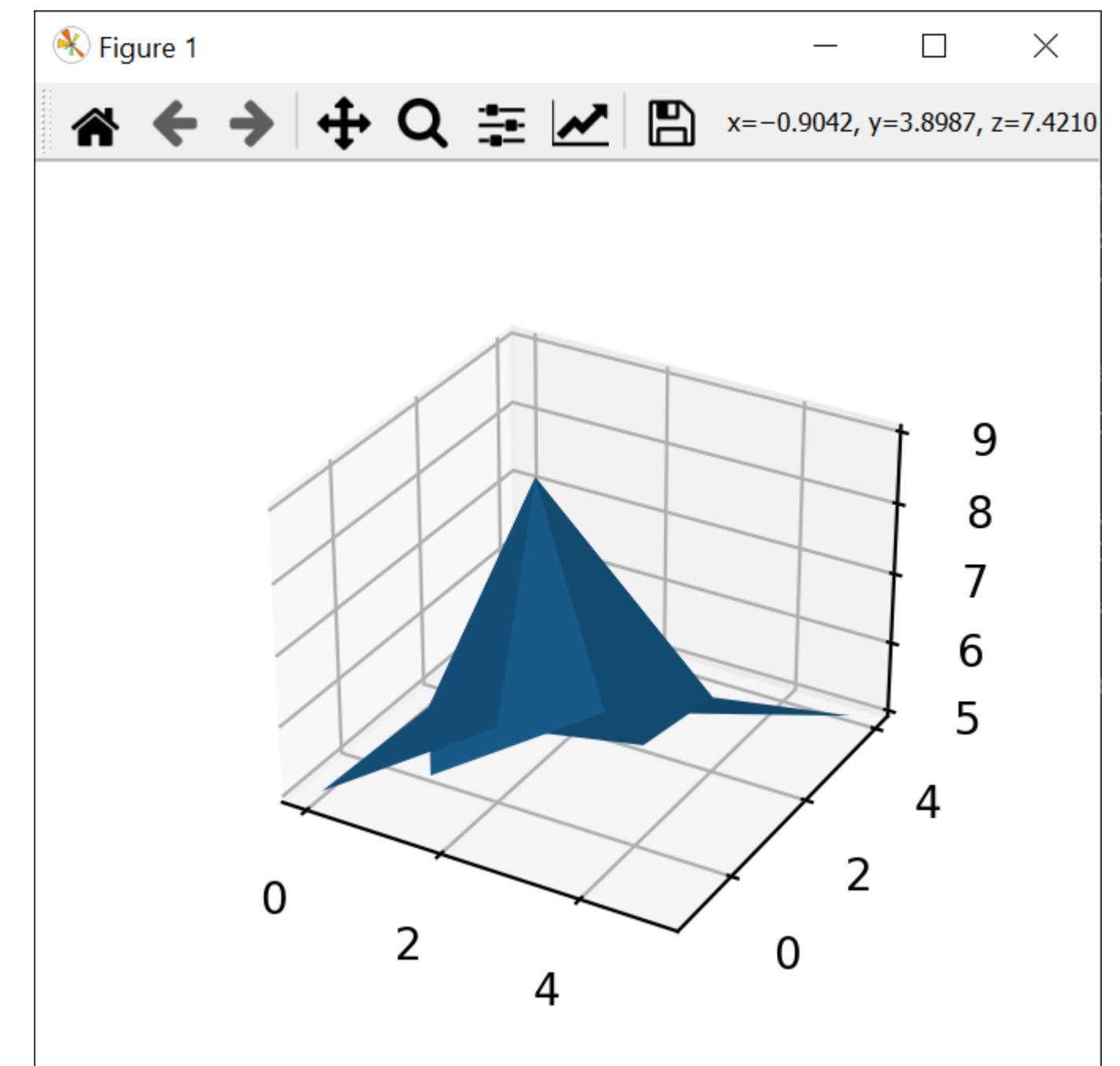
การวาดพื้นผิว 3 มิติ

```
import matplotlib.pyplot as plt
import numpy as np

ax = plt.gca(projection='3d')

X = np.array([[0, 1, 1], [2, 2, 3], [3, 4, 5]])
Y = np.array([[-1, 0, 0], [0, 1, 1], [2, 2, 4]])
Z = np.array([[5, 6, 5], [6, 9, 6], [5, 6, 5]])

ax.plot_surface(X, Y, Z)
plt.show()
```



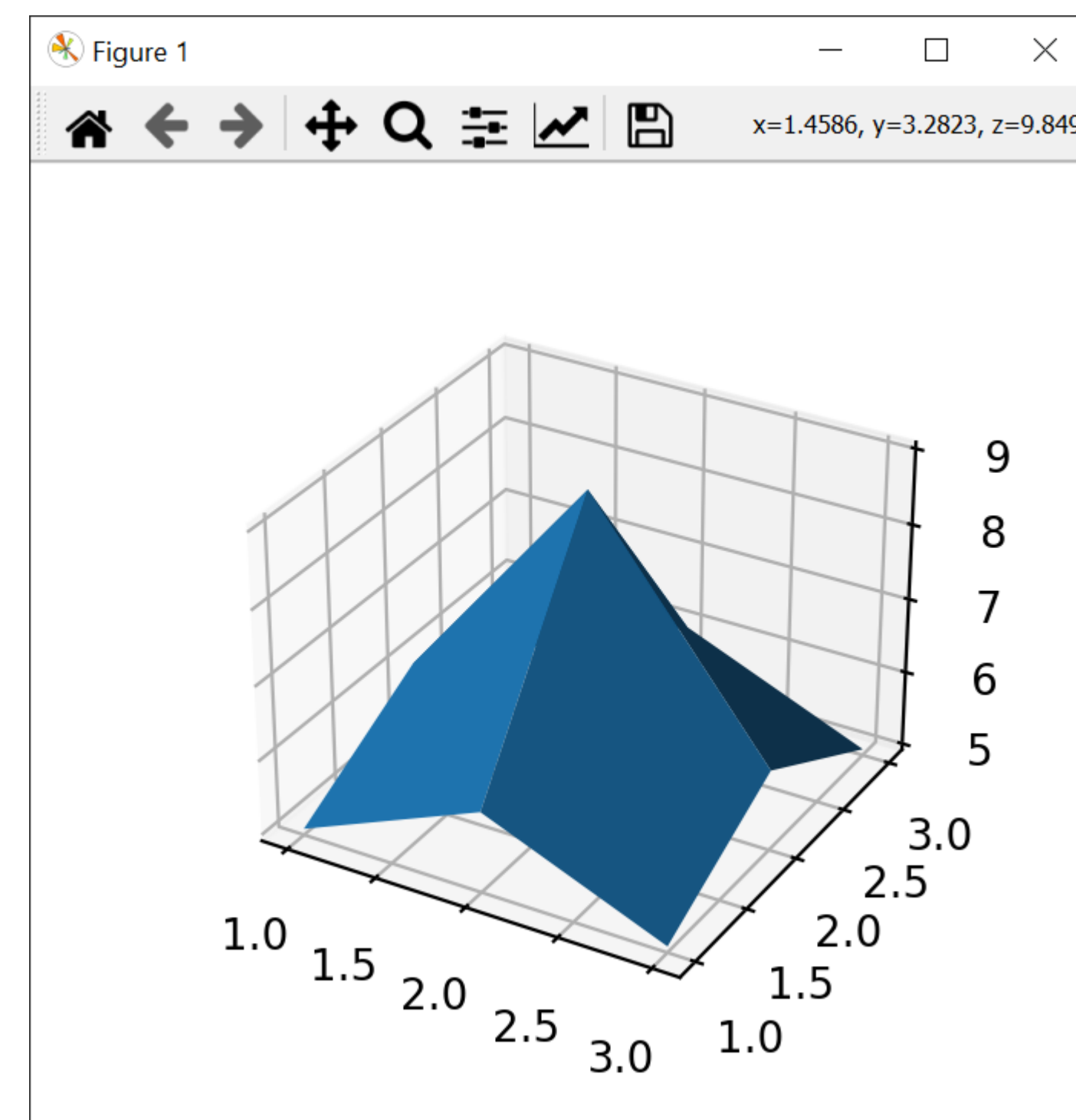


การวาดพื้นผิว 3 มิติ

```
import matplotlib.pyplot as plt
import numpy as np

ax = plt.gca(projection='3d')
X, Y = np.meshgrid(np.arange(1, 4, 1)
                    , np.arange(1, 4, 1))
Z = np.array([[5, 6, 5], [6, 9, 6], [5, 6, 5]])

ax.plot_surface(X, Y, Z)
plt.show()
```





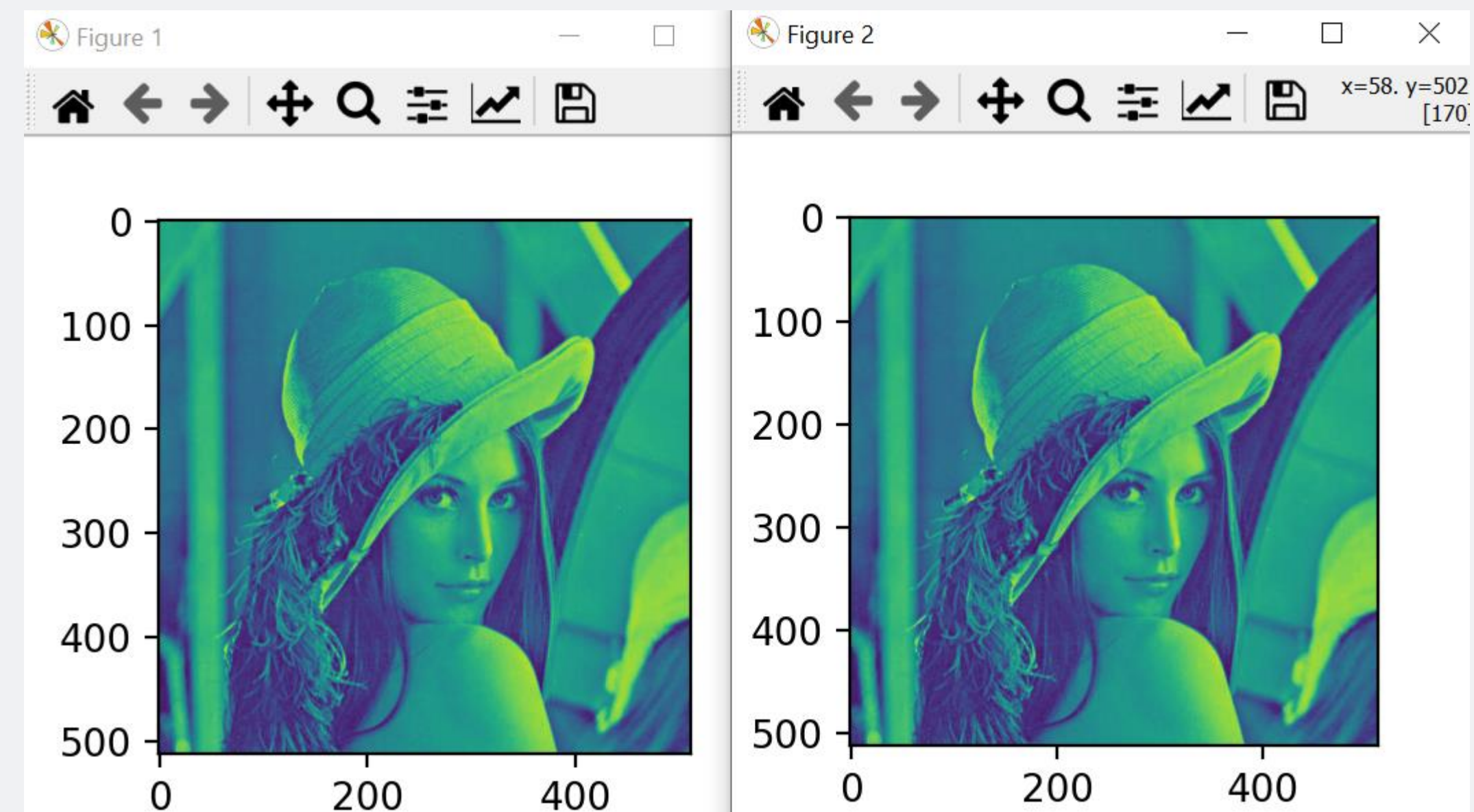
การสร้าง Figure

```
import matplotlib.pyplot as plt
import cv2

img1 = cv2.imread('lenna.jpg', 0)

plt.figure()
plt.imshow(img1)

plt.figure()
plt.imshow(img1)
plt.show()
```



ใส่อันสุดท้ายพอ ถ้าใส่มากกว่า 1 จุดมันจะค่อย ๆ แสดง



การสร้าง Subplot

```
import matplotlib.pyplot as plt
import cv2
```

```
img1 = cv2.imread('lenna.jpg', 0)
```

```
plt.figure()
plt.subplot(2,1,1)
plt.imshow(img1)
```

```
plt.subplot(2,1,2)
plt.imshow(img1)
plt.show()
```

subplot(2,2,1)

subplot(2,2,2)

subplot(2,2,3)

subplot(2,2,4)

